

TheKnife

Manuale Tecnico

Versione 1.0

9 luglio 2026

Autori:

Claudio Bonci, 759939, Sede CO

Eleonora Anna Caredda, 762576, Sede CO

Filippo Crippa, 762174, Sede CO

Matilde Lecchi, 759875, Sede CO

Università degli Studi dell'Insubria, Sede CO

Corso di Laurea in Informatica



Benvenuto/a

Il presente manuale tecnico fornisce una descrizione dettagliata dell'applicazione **TheKnife**, illustrandone la struttura interna, le scelte progettuali e le soluzioni adottate durante le fasi di analisi, progettazione e sviluppo.

Nel documento vengono presentati i principali diagrammi UML e il modello Entity-Relationship utilizzato per la progettazione del database, con l'obiettivo di descrivere in modo rigoroso l'organizzazione del sistema e le relazioni tra le sue componenti.

Sono inoltre analizzate le scelte architetturali che hanno guidato lo sviluppo dell'applicazione, le strutture dati impiegate e gli algoritmi adottati per l'implementazione delle funzionalità principali. Particolare attenzione è dedicata ai pattern di progettazione utilizzati, evidenziandone il ruolo all'interno dell'architettura software e riportando, ove opportuno, estratti di codice significativi.

Il livello di dettaglio e il linguaggio utilizzato sono rivolti a sviluppatori, progettisti software e professionisti del settore informatico interessati agli aspetti tecnici e implementativi dell'applicazione.



Indice

Benvenuto/a	i
1 Architettura sistema	1
1.1 Architettura Client	1
1.2 Architettura Server	2
1.3 Comunicazione mediante Java RMI	2
1.4 Accesso ai dati	3
1.5 Gestione della persistenza	3
1.6 Gestione delle query SQL	3
1.7 Gestione delle eccezioni	4
2 Documentazione UML	5
2.1 Use Case Diagram	6
2.2 Package Diagram	7
2.3 Class Diagram	7
2.4 Sequence Diagram - Bootstrap RMI (Connessione)	8
2.5 Sequence Diagram - Login	9
2.5.1 Sequence Diagram - Inserimento Recensione	10
2.5.2 Sequence Diagram - Risposta Recensione	11
2.5.3 Component Diagram	12
2.5.4 Deployment Diagram	13
3 Database	14
3.1 Diagramma ER	14
3.1.1 Entità	14
3.1.2 Relazioni	15
4 Scelte progettuali	17
4.1 Progettazione dell'inizializzazione automatica del database all'avvio del server	17
4.1.1 Contesto e motivazione	17
4.1.2 Obiettivo progettuale	17
4.1.3 Flusso operativo	18
4.1.4 Scelte implementative	18
4.1.5 Proprieta della soluzione	18
4.1.6 Vincoli e prerequisiti	19
4.1.7 Conclusioni	19
4.2 Separazione tra livelli del sistema	19



4.3	Utilizzo del pattern singleton	19
4.4	Utilizzo dei PreparedStatement	20
4.5	Modellazione dei privilegi tramite gerarchia degli attori	20
5	Documentazione del codice	21
5.1	Struttura Generale	21
5.1.1	Package client	21
5.1.2	Package common	21
5.1.3	Package server	21
5.2	Risorse dell'applicazione	22
5.2.1	File FXML	22
5.2.2	Foglio di stile CSS	22
5.2.3	Risorse grafiche - Il Logo	23
5.3	Librerie	23



Capitolo 1

Architettura sistema

L'applicazione **TheKnife** è stata progettata seguendo un'architettura client-server a **tre livelli** (**Three-Tier Architecture**), composta da:

- **Livello di presentazione:** implementato mediante un client desktop JavaFX
- **Livello applicativo:** costituito dal server Java RMI che espone i servizi applicativi
- **Livello dati :** rappresentato dal database PostgreSQL, accessibile esclusivamente dal server tramite JDBC

L'obiettivo principale di questa architettura è separare la logica dell'interfaccia grafica dalla logica di business e dalla gestione dei dati, migliorando modularità, manutenibilità e possibilità di estensione.

1.1 Architettura Client

Il client rappresenta il livello di presentazione dell'applicazione ed è responsabile esclusivamente della gestione dell'interfaccia grafica e dell'interazione con l'utente. L'avvio del sistema è gestito dalla classe **ClientMain**, la quale:

1. Effettua la connessione al Registry RMI
2. Recupera il riferimento remoto (stub) dell'interfaccia DBService
3. Verifica la disponibilità del server tramite una chiamata remota
4. Avvia l'applicazione JavaFX

Lo stub viene mantenuto come riferimento statico ed è accessibile dall'intera applicazione mediante il metodo: *ClientMain.getServer()* che realizza un punto di accesso centralizzato ai servizi remoti. Il client non accede mai direttamente al database, ma delega tutte le operazioni di persistenza al server attraverso invocazioni remote.



1.2 Architettura Server

Il server costituisce il livello applicativo del sistema. La classe **ServerMain** implementa l'interfaccia remota `DBService`, esportando l'oggetto remoto tramite `UnicastRemoteObject.exportObject(...)` e pubblicandolo all'interno del Registry RMI mediante `registry.rebind("TimeService", stub);`. Ogni metodo esposto dal server rappresenta un servizio applicativo accessibile dai client.

Le principali responsabilità del server sono:

- Gestione delle richieste provenienti dai client
- Validazione dei parametri ricevuti
- Costruzione delle query SQL
- Accesso al database PostgreSQL
- Trasformazione dei risultati SQL negli oggetti di dominio (Utente, Ristorante, Recensione)
- Restituzione degli oggetti serializzati al client

Il server rappresenta quindi l'unico componente autorizzato ad accedere direttamente al database.

1.3 Comunicazione mediante Java RMI

La comunicazione tra client e server utilizza il middleware Java Remote Method Invocation (RMI). Il contratto di comunicazione è definito dall'interfaccia remota `DBService`, condivisa tra client e server.

L'interfaccia espone tutte le operazioni disponibili, tra cui:

- Autenticazione
- Registrazione utenti
- Gestione ristoranti
- Gestione recensioni
- Gestione preferiti
- Gestione delle risposte

Durante una chiamata remota il flusso operativo è il seguente:

1. Il client invoca un metodo dello stub
2. RMI serializza automaticamente i parametri
3. Il server deserializza la richiesta
4. Viene eseguito il metodo corrispondente
5. Il risultato viene serializzato e restituito al client



L'utilizzo di RMI consente di nascondere completamente i dettagli della comunicazione TCP/IP, permettendo di invocare metodi remoti come normali metodi Java.

1.4 Accesso ai dati

L'accesso ai dati è implementato mediante Java Database Connectivity (JDBC). Ogni operazione sul database segue lo stesso schema operativo:

1. Apertura della connessione
2. Preparazione della query SQL
3. Associazione dei parametri
4. Esecuzione della query
5. Conversione del ResultSet negli oggetti del dominio.

Le connessioni vengono gestite mediante il costrutto try-with-resources, garantendo la chiusura automatica delle risorse anche in presenza di eccezioni.

1.5 Gestione della persistenza

Il server mantiene il database completamente trasparente rispetto al client.

Gli oggetti provenienti dal database vengono convertiti in oggetti Java appartenenti al modello condiviso:

- Utente
- Ristorante
- Recensione

Questi oggetti implementano la serializzazione richiesta da Java RMI e possono quindi essere trasferiti automaticamente tra server e client. Tale approccio evita che il client debba conoscere la struttura relazionale del database, realizzando una netta separazione tra modello relazionale e modello ad oggetti.

1.6 Gestione delle query SQL

Per tutte le operazioni di accesso al database vengono utilizzati oggetti PreparedStatement. Questa scelta progettuale offre diversi vantaggi:

- Prevenzione degli attacchi SQL Injection
- Compilazione preventiva della query
- Maggiore efficienza nelle esecuzioni ripetute
- Tipizzazione dei parametri



1.7 Gestione delle eccezioni

L'applicazione distingue due categorie di eccezioni:

- **RemoteException**: generate dal middleware RMI durante la comunicazione distribuita
- **SQLException**: generate dal driver JDBC durante l'interazione con PostgreSQL

Entrambe vengono propagate ai livelli superiori oppure intercettate per registrare informazioni diagnostiche sul server.



Capitolo 2

Documentazione UML

Questo capitolo presenta la documentazione della struttura e del comportamento del sistema tramite diagrammi UML. L'obiettivo è fornire una visione d'insieme dell'architettura del sistema a un livello di astrazione superiore rispetto al codice sorgente, facilitando la comprensione delle interazioni tra i componenti principali. La documentazione è organizzata in due aspetti complementari:

Aspetto strutturale: rappresenta le classi, i loro attributi, metodi e le relazioni statiche tra di loro
Aspetto comportamentale: illustra le interazioni dinamiche tra gli oggetti durante l'esecuzione del sistema

I diagrammi presentati di seguito sono stati selezionati per rappresentare chiaramente il funzionamento del sistema, senza includere elementi ridondanti. Ogni diagramma contribuisce a illustrare aspetti specifici dell'architettura client/server implementata.

2.1 Use Case Diagram



Lo Use Case Diagram rappresenta le interazioni tra gli attori del sistema e le funzionalità offerte dall'applicazione TheKnife. Gli attori sono organizzati tramite una gerarchia di generalizzazione che riflette i diversi ruoli e livelli di accesso.

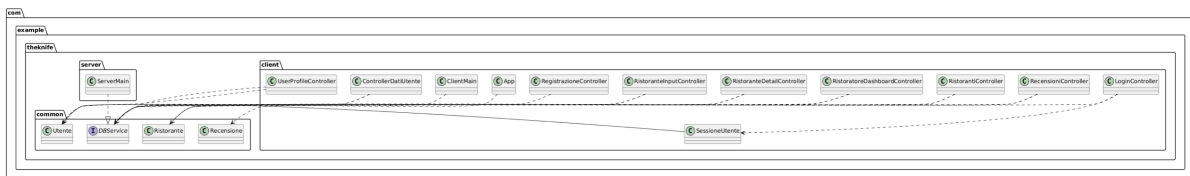
L'Ospite può consultare i ristoranti, visualizzare i dettagli e leggere le recensioni, oltre ad accedere alle funzionalità di registrazione e login. L'Utente registrato (cliente) eredita tutte le funzionalità dell'ospite e può inserire, modificare e cancellare recensioni, gestire i ristoranti preferiti e aggiornare il proprio profilo. Il Ristoratore eredita le funzionalità



dell'utente registrato e dispone di operazioni aggiuntive, come l'inserimento e la gestione dei propri ristoranti e la possibilità di rispondere alle recensioni ricevute.

Le operazioni che richiedono autenticazione includono il caso d'uso "Autenticazione" tramite la relazione «include», evidenziando che ogni funzione protetta necessita di una sessione valida. La struttura gerarchica degli attori permette di modellare in modo chiaro il controllo degli accessi e la gestione dei privilegi all'interno del sistema.

2.2 Package Diagram



Il Package Diagram descrive l'organizzazione logica dei componenti software del sistema TheKnife, suddivisi nei package common, client e server.

Il package common contiene gli elementi condivisi tra client e server: l'interfaccia remota DBService, che definisce il contratto delle operazioni disponibili tramite RMI, e le classi del modello Utente, Ristorante e Recensione, utilizzate come oggetti serializzabili per lo scambio dei dati.

Il package client contiene tutti i componenti responsabili dell'interazione con l'utente: l'applicazione JavaFX (App), la classe di bootstrap ClientMain, il servizio RMIService, la gestione della sessione tramite SessioneUtente e tutti i controller dell'interfaccia grafica (Login, Registrazione, Ristoranti, Dettaglio, Recensioni, Dashboard, Input, Profilo, DatiUtente). Tutti i controller dipendono dall'interfaccia DBService, garantendo la separazione tra logica di presentazione e logica applicativa lato server.

Il package server contiene la classe ServerMain, che implementa l'interfaccia DBService e gestisce tutte le operazioni di accesso ai dati tramite JDBC. Il server rappresenta l'unico punto autorizzato a comunicare con il database PostgreSQL.

2.3 Class Diagram



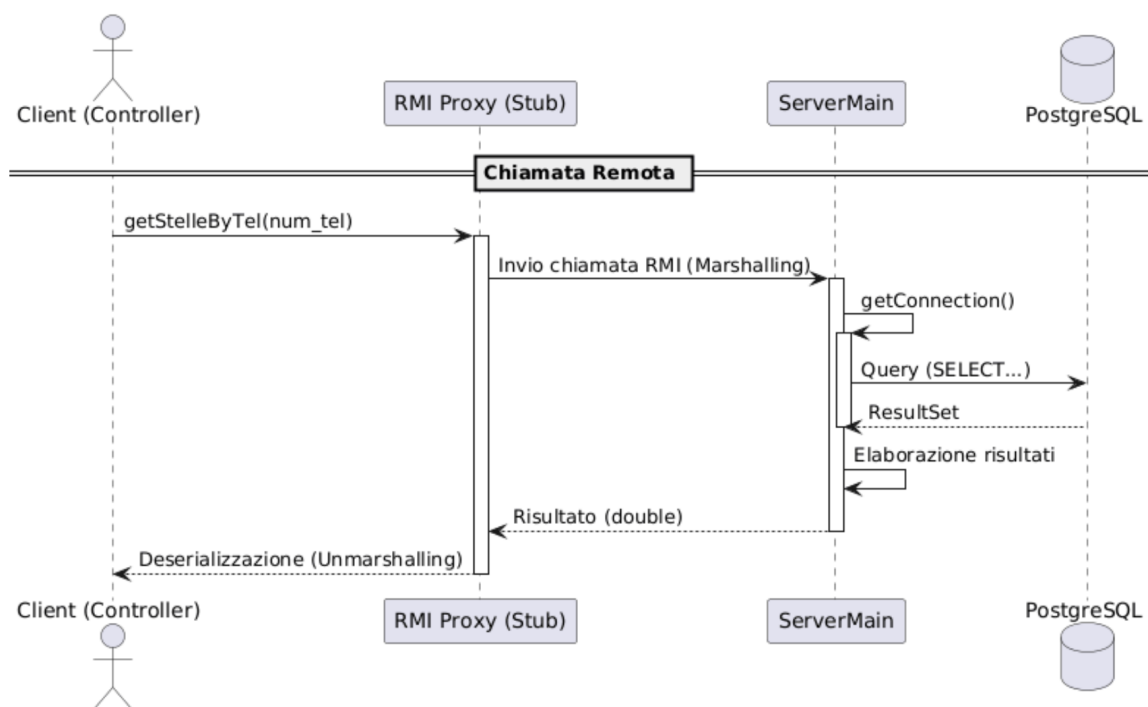


Il Class Diagram descrive la struttura statica del sistema TheKnife, evidenziando le principali classi, interfacce e relazioni. L'interfaccia DBService rappresenta il contratto di comunicazione tra client e server e include operazioni per la gestione degli utenti, dei ristoranti, delle recensioni, dei preferiti e delle risposte.

La classe ServerMain implementa l'interfaccia remota e fornisce l'implementazione concreta delle operazioni richieste dai client, gestendo la connessione al database e l'esecuzione delle query SQL. I controller del package client gestiscono la logica applicativa e dipendono esclusivamente dall'interfaccia DBService, applicando il principio di inversione delle dipendenze.

Le classi del modello (Utente, Ristorante, Recensione) rappresentano le entità principali del sistema e vengono utilizzate per lo scambio dei dati tra client e server. La classe SessioneUtente mantiene lo stato dell'utente autenticato durante la sessione corrente.

2.4 Sequence Diagram - Bootstrap RMI (Connessione)



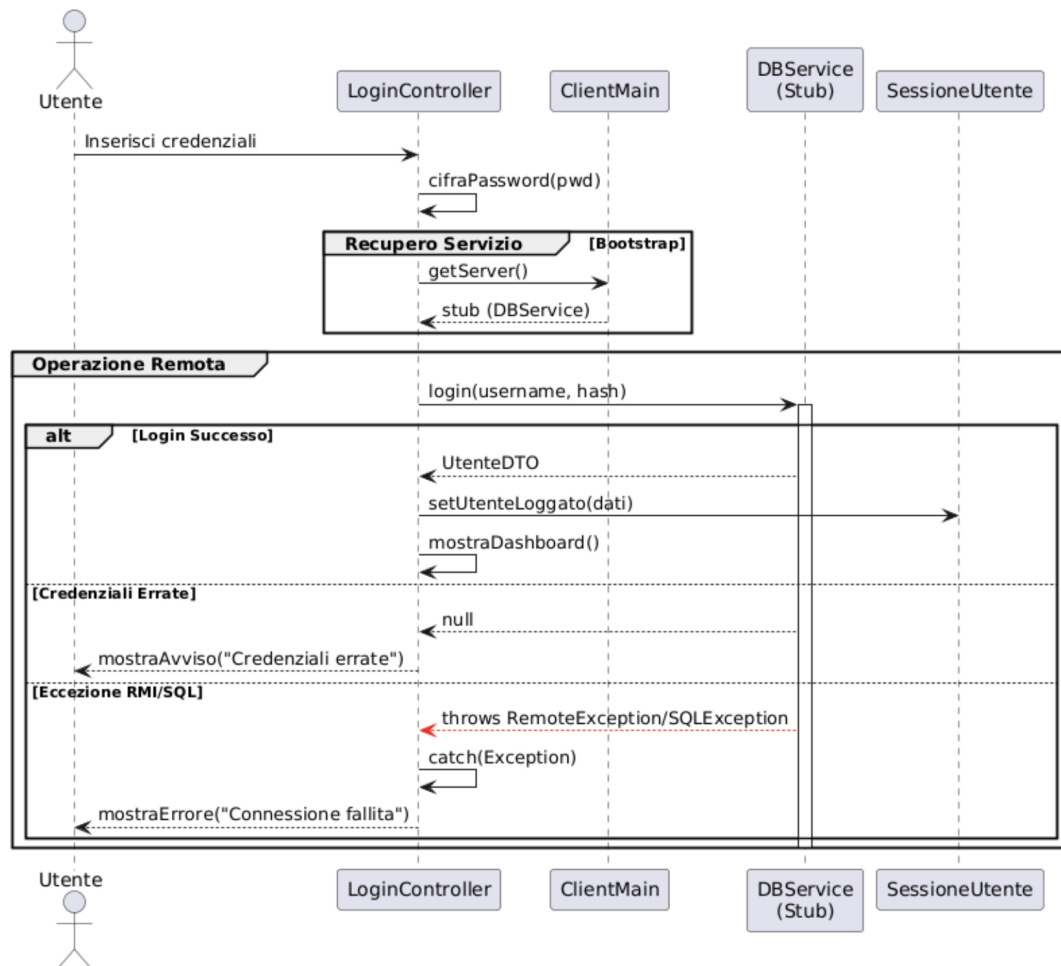
Il diagramma di sequenza relativo al bootstrap RMI descrive il processo di inizializzazione della comunicazione remota tra client e server. Il controller del client invoca un metodo remoto tramite lo stub dell'interfaccia DBService. Lo stub serializza la richiesta (marshalling) e la inoltra al server tramite il middleware RMI.

ServerMain riceve la chiamata, deserializza i parametri (unmarshalling) ed esegue l'operazione richiesta, che può includere l'interrogazione del database tramite JDBC. Il risultato viene restituito al client seguendo il processo inverso di serializzazione. Il diagramma



evidenzia la separazione tra logica di presentazione, comunicazione remota e persistenza dei dati, garantendo un'architettura modulare e facilmente estendibile.

2.5 Sequence Diagram - Login

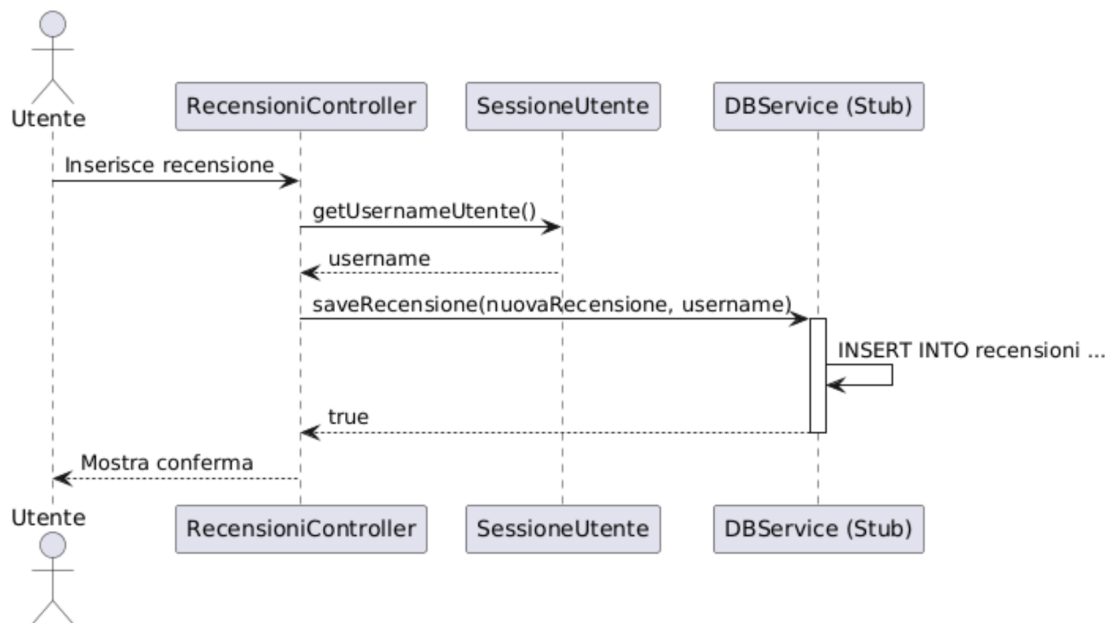


Il diagramma di sequenza del login descrive il processo di autenticazione dell'utente all'interno del sistema TheKnife, evidenziando la comunicazione remota tra client e server tramite RMI. Il flusso inizia con l'inserimento delle credenziali da parte dell'utente, gestito dal LoginController. Prima di inviare i dati al server, il controller applica la funzione di cifratura `cifraPassword(pwd)`, garantendo che la password non venga mai trasmessa in chiaro.

Il controller richiede a ClientMain il riferimento al servizio remoto tramite `getServer()`, ottenendo lo stub dell'interfaccia DBService. L'operazione remota `login(username, hash)` viene invocata sullo stub, che serializza la richiesta e la inoltra al server. ServerMain deserializza i parametri, verifica le credenziali interrogando il database e restituisce l'esito dell'autenticazione.

Se l'autenticazione ha successo, il controller aggiorna la sessione tramite `SessioneUtente.setUtenteLoggato()`, rendendo disponibili le informazioni dell'utente per le operazioni successive. In caso di credenziali errate viene mostrato un messaggio di errore, mentre eventuali problemi di comunicazione o accesso ai dati (`RemoteException`, `SQLException`) vengono gestiti tramite messaggi informativi all'utente. Il diagramma evidenzia l'uso del pattern Proxy tipico di RMI e la separazione tra logica di presentazione, comunicazione remota e gestione dei dati.

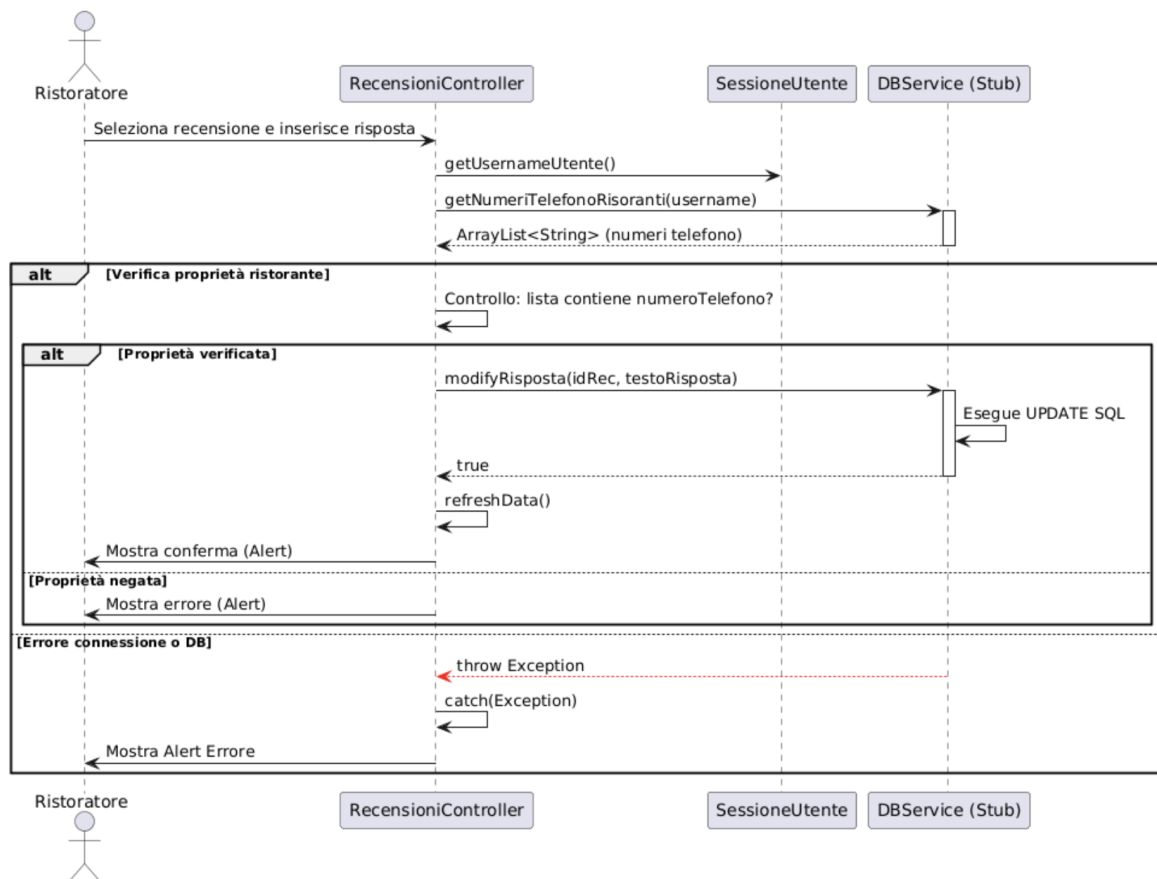
2.5.1 Sequence Diagram - Inserimento Recensione



Il diagramma di sequenza relativo all'inserimento di una recensione descrive il flusso che consente a un utente autenticato di pubblicare un nuovo contenuto. L'utente inserisce i dati della recensione tramite l'interfaccia grafica, e il **RecensioniController** recupera l'identità dell'utente tramite `SessioneUtente.getUsernameUtente()`, assicurando che la recensione venga correttamente associata al suo autore.

Il controller invoca il metodo remoto `saveRecensione(nuovaRecensione, username)` sullo stub **DBService**. Il server esegue la query SQL di inserimento nella tabella delle recensioni e restituisce un valore booleano che indica il successo dell'operazione. Il controller gestisce l'esito mostrando un messaggio di conferma o di errore. Il diagramma evidenzia la separazione tra logica applicativa, gestione della sessione e persistenza dei dati, riflettendo fedelmente il comportamento implementato nel sistema.

2.5.2 Sequence Diagram - Risposta Recensione

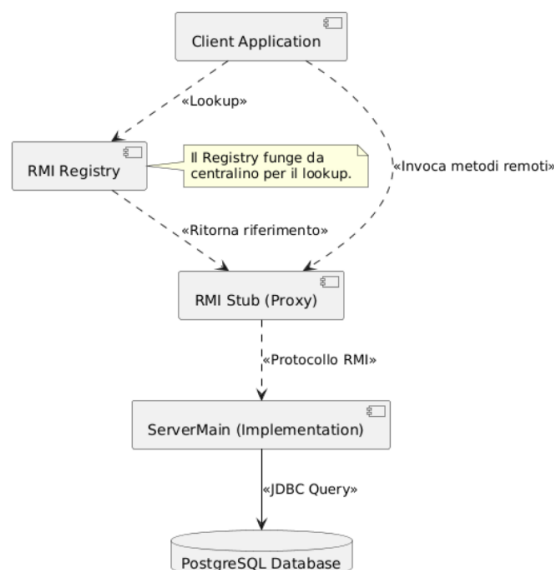


Il diagramma di sequenza relativo all'inserimento di una risposta da parte del ristorante descrive il flusso che consente di aggiungere un commento ufficiale a una recensione. Il ristorante inserisce il testo della risposta tramite l'interfaccia grafica, e il RecensioniController recupera lo username dell'utente autenticato tramite SessioneUtente.getUsernameUtente().

Prima di procedere con la modifica, il controller richiede al servizio remoto la lista dei ristoranti associati al ristorante tramite getNumeriTelefonoRistoranti(username). Questo passaggio permette di verificare che il ristorante della recensione appartenga effettivamente all'utente che sta tentando di rispondere. Solo in caso di esito positivo viene invocata l'operazione remota modifyRisposta(idRec, testo) sullo stub DBService, che esegue l'aggiornamento nel database.

Se il controllo dei permessi fallisce, il sistema impedisce l'operazione per motivi di sicurezza. Il diagramma evidenzia la separazione tra logica applicativa, autorizzazione e persistenza, garantendo che ogni ristorante possa rispondere esclusivamente alle recensioni dei propri ristoranti.

2.5.3 Component Diagram



Il Component Diagram descrive la suddivisione logica del sistema TheKnife nei principali componenti software e le relazioni tra essi. L'architettura segue un modello client-server basato su RMI, nel quale il client utilizza un servizio remoto esposto dal server per accedere alle funzionalità applicative.

Il componente Client Application contiene la logica di presentazione e di interazione con l'utente. I controller JavaFX gestiscono le operazioni dell'interfaccia e comunicano con il server esclusivamente tramite l'interfaccia remota DBService, ottenuta attraverso il servizio RMIService. Questo approccio garantisce che il client non dipenda dall'implementazione concreta del server, ma da un contratto astratto.

Il componente Common contiene le classi condivise tra client e server: l'interfaccia DBService, che definisce le operazioni disponibili tramite RMI, e le classi del modello (Utente, Ristorante, Recensione) utilizzate per lo scambio dei dati. Questo package funge da ponte tra i due livelli dell'applicazione.

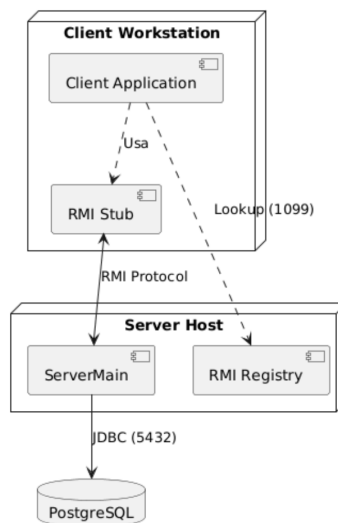
Il componente ServerMain rappresenta il livello applicativo lato server. Implementa l'interfaccia DBService e gestisce tutte le operazioni di business e di persistenza. Il server comunica con il database PostgreSQL tramite JDBC, eseguendo query SQL e restituendo i risultati ai client attraverso il meccanismo RMI.

Il componente Database Server contiene il database PostgreSQL, che memorizza tutte le informazioni persistenti dell'applicazione. Il database è accessibile esclusivamente dal server, garantendo sicurezza e separazione tra i livelli.

Il diagramma evidenzia chiaramente la separazione tra presentazione, logica applicativa e persistenza, oltre all'utilizzo del pattern Proxy tipico delle architetture RMI.



2.5.4 Deployment Diagram



Il Deployment Diagram rappresenta la distribuzione fisica dei componenti del sistema TheKnife sui diversi nodi di esecuzione. L'architettura adottata è una struttura client-server a tre livelli, che separa la logica di presentazione, la logica applicativa e la persistenza dei dati.

Il nodo Client Machine ospita l'applicazione JavaFX, che gestisce l'interazione con l'utente. Il client include i controller dell'interfaccia e il servizio RMIService, responsabile del recupero dello stub remoto tramite il metodo lookup("DBService"). Il client non accede direttamente al database, ma comunica esclusivamente con il server tramite RMI.

Il nodo Server Machine ospita il server applicativo, rappresentato dalla classe ServerMain. Il server implementa l'interfaccia remota DBService, gestisce le richieste provenienti dai client e applica le regole di business. Ogni operazione remota viene eseguita lato server, che si occupa della validazione dei parametri, dell'esecuzione delle query SQL e della trasformazione dei risultati in oggetti del dominio.

Il nodo Database Server ospita il database PostgreSQL, che contiene lo schema dati dell'applicazione. Il server comunica con il database tramite JDBC, garantendo che la persistenza sia completamente isolata dal client. Questo modello aumenta la sicurezza, poiché nessun client può accedere direttamente ai dati.

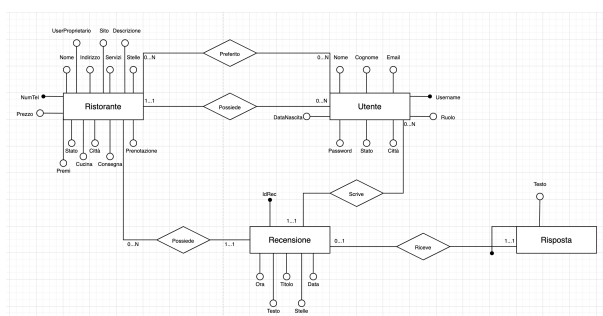
Il diagramma evidenzia il ruolo del middleware RMI nel collegamento tra client e server, e mostra come la distribuzione dei componenti garantisca modularità, sicurezza e scalabilità.



Capitolo 3

Database

3.1 Diagramma ER



La foto rappresenta il diagramma ER tramite il quale è stato sviluppato il database di **TheKnife**.

Il diagramma ER adottato per la progettazione del database di TheKnife rappresenta in modo rigoroso le entità fondamentali del dominio applicativo e le relazioni che intercorrono tra esse. La struttura è stata modellata per riflettere fedelmente le dinamiche reali del sistema, garantendo coerenza, integrità dei dati e semplicità di gestione.

Dal punto di vista della progettazione, il modello risulta normalizzato privo di ridondanze: ogni informazione è collocata nell'entità più appropriata e non esistono dipendenze transitive o attributi multivalore. La scelta delle chiavi primarie è stata effettuata in modo da garantire identificatori univoci e significativi: Username per l'utente, NumTel per il ristorante e un identificatore artificiale per la recensione. Questa impostazione permette di mantenere un database semplice da interrogare e coerente con le esigenze dell'applicazione.

3.1.1 Entità

Il modello si basa su quattro entità principali ciascuna delle quali svolge un ruolo specifico all'interno dell'applicazione: **Utente**, **Ristorante**, **Recensione** e **Risposta**.



Utente

L'entità **Utente** rappresenta ogni persona registrata sulla piattaforma, che può assumere il ruolo di cliente o di proprietario. Gli attributi di **Utente**, sono nove: Nome, Cognome, Email, Username, Ruolo, Città, Stato, DataNascita, Password. Tra questi, **Username** è la chiave primaria.

Ristorante

L'entità **Ristorante** raccoglie tutte le informazioni relative ai locali presenti nell'applicazione: dati anagrafici, tipologia di cucina, servizi offerti, fascia di prezzo e altri attributi utili alla consultazione. Ogni ristorante è associato a un unico proprietario, mentre la chiave primaria è **NumTel**, ovvero il numero di telefono del ristorante che è univoco per ogni ristorante.

Recensione

L'entità **Recensione** rappresenta il contenuto generato dagli utenti. Ogni recensione è collegata sia all'utente che l'ha scritta sia al ristorante a cui si riferisce. Questo duplice legame permette di tracciare in modo preciso l'origine e la destinazione del contenuto, assicurando trasparenza e integrità dei dati.

Risposta

A completare il modello vi è l'entità **Risposta**, che consente ai proprietari dei ristoranti di replicare alle recensioni ricevute. Ogni recensione può avere al massimo una risposta, riflettendo un comportamento tipico delle piattaforme di recensioni: un singolo commento ufficiale per ogni feedback ricevuto.

3.1.2 Relazioni

Le relazioni presenti nel modello descrivono in modo naturale le operazioni consentite dal sistema.

Relazione: Possiede

Le entità coinvolte sono Utente e Ristorante. Ogni ristorante è gestito da un unico proprietario (utente con ruolo "ristoratore"). Un utente ristoratore può gestire uno o più ristoranti. Le cardinalità sono (0,N) da Utente a (1,1) da Ristorante.

Relazione: Scrive

Le entità coinvolte sono Ristorante e Recensione. Un ristorante riceve una o più recensioni e ogni recensione si riferisce ad un unico ristorante. La cardinalità è (1,N) da Ristorante a (1,1) da Recensione.

Relazione: Riceve (Recensione-Risposta)

Le entità coinvolte sono Recensione e Risposta. Una recensione può ricevere al massimo una risposta dal proprietario del ristorante e ogni risposta è legata ad una sola recensione.



La cardinalità è (0,1) da Recensione a (0,1) da Risposta. Limitare una risposta per recensione riflette il modello standard delle piattaforme di recensioni, dove un proprietario fornisce un singolo commento ufficiale per ogni feedback ricevuto.

Relazione: Preferito

Le entità coinvolte sono Utente e Ristorante. Un utente può aggiungere uno o più ristoranti ai propri preferiti ed un ristorante può essere aggiunto ai preferiti da zero o più utenti. La cardinalità è (0,N) da Utente a (0,N) da Ristorante.



Capitolo 4

Scelte progettuali

Le scelte progettuali adottate nello sviluppo di TheKnife mirano a garantire un sistema modulare, sicuro, facilmente manutenibile e coerente dal punto di vista dell'interfaccia grafica. Di seguito vengono illustrate le decisioni principali che hanno guidato la realizzazione dell'applicazione.

4.1 Progettazione dell'inizializzazione automatica del database all'avvio del server

4.1.1 Contesto e motivazione

Nelle applicazioni client-server basate su persistenza relazionale, la disponibilità del database rappresenta una preconditione necessaria per l'erogazione dei servizi applicativi. In assenza di una strategia di bootstrap, la prima esecuzione del sistema richiede attività manuali (creazione del database, import dello schema, caricamento dei dati iniziali) che aumentano il rischio di errore operativo e riducono la riproducibilità dell'ambiente.

Per mitigare tali criticità, è stata adottata una procedura di inizializzazione automatica eseguita all'avvio del server, con l'obiettivo di garantire che il database applicativo sia disponibile prima della pubblicazione dei servizi remoti.

4.1.2 Obiettivo progettuale

La soluzione implementata persegue i seguenti obiettivi:

1. verificare in modo deterministico l'esistenza del database applicativo;
2. creare il database in caso di assenza;
3. ripristinare schema e dati iniziali a partire da uno script SQL versionato;
4. consentire al server di proseguire l'avvio solo dopo il completamento del bootstrap.



4.1.3 Flusso operativo

Il flusso di inizializzazione puo essere formalizzato come segue:

1. Avvio del processo server.
2. Tentativo di connessione JDBC al database applicativo.
3. Se la connessione ha esito positivo, la fase di bootstrap viene terminata.
4. Se la connessione fallisce con codice `SQLState 3D000` (database inesistente), viene avviata la procedura di creazione:
 - connessione al database di sistema `postgres`;
 - esecuzione dell'istruzione `CREATE DATABASE`;
 - esecuzione dello script di inizializzazione SQL.
5. In caso di successo, il server completa la fase di avvio e registra i servizi RMI.
6. In caso di errore non recuperabile, il server registra l'anomalia e interrompe il bootstrap.

4.1.4 Scelte implementative

La procedura adotta una distinzione esplicita tra:

- **database target** (applicativo), oggetto della verifica di esistenza;
- **database di amministrazione** (`postgres`), usato esclusivamente per operazioni DDL di creazione.

L'inizializzazione strutturale e delegata a uno script SQL esterno incluso nelle risorse applicative, cosi da ottenere:

- tracciabilita delle modifiche allo schema;
- riuso della stessa base dati iniziale in ambienti differenti;
- separazione tra logica applicativa e logica di provisioning.

L'esecuzione dello script tramite `psql` garantisce compatibilita con dump PostgreSQL completi (DDL, vincoli, indici, dati), riducendo la complessita di parsing lato applicazione.

4.1.5 Proprieta della soluzione

Dal punto di vista ingegneristico, la soluzione presenta le seguenti proprieta:

- **Idempotenza condizionata:** se il database esiste, il bootstrap non altera lo stato.
- **Riproducibilita:** la stessa sorgente SQL produce configurazioni coerenti tra installazioni.
- **Riduzione del debito operativo:** vengono eliminati passaggi manuali di setup iniziale.
- **Osservabilita:** l'output della procedura e tracciato a log per diagnosi e audit.



4.1.6 Vincoli e prerequisiti

L'approccio richiede:

- accessibilit  dell'istanza PostgreSQL;
- disponibilit  del client `psql` nel `PATH` del sistema;
- credenziali con privilegi sufficienti alla creazione di database;
- coerenza tra versione del dump e versione del motore PostgreSQL.

In assenza di tali prerequisiti, la procedura non pu  concludersi positivamente e deve essere gestita con opportune politiche di fail-fast o recovery assistito.

4.1.7 Conclusioni

L'inizializzazione automatica del database all'avvio del server costituisce una scelta architetturale coerente con i principi di robustezza, automazione e riproducibilit . Essa migliora la qualit  del ciclo di rilascio, riduce la probabilit  di errori umani nelle prime installazioni e standardizza il comportamento del sistema nei diversi contesti di esecuzione (sviluppo, test, demo, pre-produzione).

4.2 Separazione tra livelli del sistema

L'applicazione   stata progettata seguendo una chiara separazione tra:

- Interfaccia grafica
- Modello condiviso
- Logica applicativa lato server

Questa suddivisione riflette una scelta progettuale fondamentale: isolare i ruoli dei componenti per ottenere un sistema pi  sicuro e pi  semplice da mantenere.

4.3 Utilizzo del pattern singleton

Per garantire un accesso centralizzato e controllato ad alcune componenti dell'applicazione,   stato adottato il **pattern Singleton**. L'uso del Singleton permette di:

- Evitare duplicazioni di stato
- Garantire coerenza tra le schermate,
- Semplificare la gestione delle risorse condivise
- Ridurre il rischio di errori dovuti a istanze multiple non sincronizzate



4.4 Utilizzo dei PreparedStatement

Per la gestione delle operazioni SQL, l'applicazione utilizza principalmente oggetti PreparedStatement, che rappresentano una scelta progettuale orientata alla sicurezza e alla robustezza del sistema. I PreparedStatement sono stati impiegati per:

- Prevenire attacchi SQL injection
- Tipizzare correttamente i parametri
- Migliorare la leggibilità delle query
- Gestire in modo sicuro l'inserimento e la modifica dei dati

Tuttavia, non tutte le query dell'applicazione utilizzano PreparedStatement: alcune operazioni di lettura più semplici o prive di parametri dinamici sono state implementate senza PreparedStatement.

Questa scelta è stata adottata per mantenere il codice più snello nelle parti dove la complessità non lo giustificava, senza compromettere la sicurezza delle operazioni sensibili.

4.5 Modellazione dei privilegi tramite gerarchia degli attori

Gli attori sono: Ospite, Cliente, Ristoratore. Questa struttura non rappresenta una semplice scala di “più o meno privilegi”, ma una differenziazione funzionale: ogni attore dispone di un insieme di operazioni progettato per rispecchiare il suo ruolo reale nell'applicazione, opportunamente descritte nel Manuale Utente.

La gerarchia degli attori è stata progettata per:

- Rappresentare fedelmente i ruoli reali dell'applicazione
- Semplificare la gestione dei permessi
- Evitare controlli ridondanti nei controller
- Garantire che ogni operazione sia eseguibile solo dagli attori autorizzati
- Mantenere coerenza tra i casi d'uso e il modello ER

In questo modo l'applicazione rimane chiara, coerente e facilmente estendibile.



Capitolo 5

Documentazione del codice

5.1 Struttura Generale

Il progetto è organizzato in tre package principali:

```
com.example.theknife.client  
com.example.theknife.common  
com.example.theknife.server
```

5.1.1 Package client

Contiene tutta la logica lato client, inclusi:

- Applicazione JavaFX
- Controller delle schermate
- Gestione della sessione
- Servizio RMI

5.1.2 Package common

Contiene gli elementi condivisi tra client e server:

- DBService.java: interfaccia remota RMI che definisce tutte le operazioni disponibili
- Utente.java, Ristorante.java, Recensione.java : classi del dominio serializzabili

Questa scelta progettuale garantisce compatibilità RMI e modello dati condiviso.

5.1.3 Package server

Contiene la logica lato server, ServerMain.java implementa DBService e gestisce:

- Connessione JDBC
- Costruzione delle query



- Validazione dei parametri
- Conversione dei risultati SQL in oggetti Java
- Pubblicazione del servizio RMI

È l'unico punto autorizzato ad accedere al database.

5.2 Risorse dell'applicazione

Oltre al codice sorgente Java, l'applicazione TheKnife utilizza un insieme di risorse esterne che contribuiscono alla definizione dell'interfaccia grafica e dello stile visivo.

5.2.1 File FXML

I file FXML definiscono la struttura dell'interfaccia grafica dell'applicazione. Ogni schermata è descritta tramite un file XML che specifica tutto il necessario per la visualizzazione corretta delle informazioni. Ogni FXML è associato a un controller dedicato nel package client.

5.2.2 Foglio di stile CSS

Il file **stile.css** definisce l'aspetto grafico dell'intera applicazione TheKnife e rappresenta il punto centrale del sistema di design dell'interfaccia.

La scelta di utilizzare un foglio di stile dedicato permette di separare completamente la logica applicativa dalla presentazione, garantendo uniformità visiva tra tutte le schermate e facilitando eventuali modifiche estetiche future.

Il foglio di stile è organizzato in sezioni tematiche e utilizza un approccio moderno basato su design tokens, ovvero variabili globali dichiarate nella radice (.root) e riutilizzate in tutto il CSS. Questi token definiscono:

- Palette colori primaria e neutra
- Font e dimensioni tipografiche
- Raggi di arrotondamento
- Ombre e profondità
- Accenti grafici dedicati (es. pulsante Google Maps)

Grazie a questa struttura, modificare un colore o un valore tipografico nella sezione dei design tokens consente di aggiornare automaticamente l'intero tema grafico dell'applicazione.

Il resto del foglio di stile è suddiviso in blocchi che definiscono gli elementi principali dell'interfaccia:

- **Card di login e contenitori:** pannelli con ombre, bordi arrotondati e layout coerente
- **Campi di testo e form:** gestione dello stato di focus, bordi, padding e leggibilità



- **Pulsanti:** varianti primarie, secondarie, ghost, danger, navigazione e pulsanti dedicati (Maps), ciascuno con comportamento hover/pressed
- **Etichette e testi informativi:** gerarchie tipografiche per titoli, sottotitoli, descrizioni e messaggi di servizio
- componenti complessi come TableView, ListView, grafici e sezioni statistiche, con stili dedicati per header, celle, righe alternate, scrollbar e palette dei grafici
- pagine specifiche come profilo utente, dashboard del ristoratore, dettaglio ristorante e recensioni, ognuna con pannelli, box informativi e layout personalizzati

L'insieme di queste regole CSS permette di ottenere un'interfaccia moderna, coerente e professionale, mantenendo al tempo stesso una struttura facilmente estendibile e modificabile.

5.2.3 Risorse grafiche - Il Logo

Viene utilizzata una singola immagine: il logo dell'applicazione.



5.3 Librerie

L'applicazione TheKnife fa uso di un insieme di librerie esterne che supportano la gestione dell'interfaccia grafica, la comunicazione remota, l'accesso ai dati e le operazioni di testing. Queste dipendenze costituiscono la base tecnologica del progetto e permettono di integrare funzionalità avanzate mantenendo al tempo stesso un'architettura modulare, sicura e facilmente manutenibile.

L'elenco seguente riporta tutte le librerie utilizzate nel progetto, con le relative versioni:

- org.openjfx:javafx-controls:\${javadoc.version}
- org.openjfx:javafx-fxml:\${javadoc.version}
- org.openjfx:javafx-graphics:\${javadoc.version}
- org.openjfx:javafx-base:\${javadoc.version}
- org.controlsfx:controlsfx:11.2.1
- org.postgresql:postgresql:42.7.4
- org.junit.jupiter:junit-jupiter-api:\${junit.version} (scope: test)
- org.junit.jupiter:junit-jupiter-engine:\${junit.version} (scope: test)